

MCP Maker

Specification MVP - Generateur de serveurs MCP depuis des repositories GitHub

Version 0.1 - 3 juillet 2026

Vision courte

Transformer une codebase existante en serveur MCP production-ready, sans promettre une magie automatique a 100 %. Le produit analyse, propose, securise, genere, teste et documente. Le developpeur garde toujours le dernier mot avant generation.

1. Pourquoi ce MVP a du sens

Le Model Context Protocol (MCP) est un standard ouvert qui permet a des applications IA de se connecter a des systemes externes: fichiers, bases de donnees, APIs, outils, workflows et prompts specialises. Pour un developpeur, le probleme n'est pas seulement d'ecrire un serveur MCP: le vrai probleme est de choisir quelles capacites exposer, comment les documenter, comment limiter les risques, puis comment produire un serveur fiable et installable dans les clients IA.

L'opportunite de MCP Maker est donc de devenir un assistant de transformation controlee: il ne remplace pas le jugement du developpeur, il accelere l'analyse du repo, propose des tools MCP, genere le code et verifie que le resultat tient debout.

2. Definition du MVP

Objectif MVP:

- L'utilisateur colle une URL GitHub publique.
- L'application analyse le repo et detecte le stack, les routes, services, scripts et schemas importants.
- L'application propose une liste de tools MCP possibles avec une description, un schema d'entree/sortie et un niveau de risque.
- L'utilisateur choisit les tools qu'il veut exposer.
- Le systeme genere un serveur MCP TypeScript, avec README, .env.example, tests de base et configuration client.
- L'utilisateur telecharge le projet genere sous forme de ZIP ou le pousse vers un repository cible dans une version ulterieure.

Inclus dans le MVP	Hors MVP initial
GitHub public + repos JS/TS en priorite	Tous les langages et tous les frameworks
Generation serveur MCP TypeScript	Generation multi-langage complete des le debut
Selection humaine des tools	Generation automatique sans validation humaine
Security scoring simple et explicable	Sandbox enterprise avancee / IAM complet
ZIP telechargeable	Marketplace MCP et deploiement cloud complet

3. Features principales du MVP

F1 - Import GitHub et creation du projet

Description: L'utilisateur fournit l'URL d'un repository GitHub. L'application recupere les metadonnees publiques, clone le repo dans un espace temporaire et cree un projet d'analyse.

Valeur utilisateur: Demarrer rapidement sans configuration complexe.

Criteres d'acceptation MVP:

- URL valide detectee
- Repo clone sans exposer les secrets
- Arbre de fichiers disponible pour l'analyse

F2 - Analyse statique de la codebase

Description: Le systeme lit les fichiers structurants: package.json, tsconfig, prisma/schema.prisma, app routes, controllers, services, README, scripts npm et dossiers importants. Il identifie le framework et les conventions du projet.

Valeur utilisateur: Comprendre le projet avant de generer quoi que ce soit.

Criteres d'acceptation MVP:

- Stack detecte avec confiance
- Fichiers cles listes
- Résumé technique court produit

F3 - Decouverte des capacites exposables

Description: A partir des routes, services, schemas et scripts, l'application extrait des capacites candidates: lecture de donnees, recherche, creation, mise a jour, export, execution de tests, inspection de schema, etc.

Valeur utilisateur: Transformer la logique existante en opportunités de tools MCP.

Criteres d'acceptation MVP:

- Chaque capacite a un nom lisible
- Chaque capacite reference le fichier source
- Chaque capacite indique son type: read, write, destructive ou system

F4 - Designer de tools MCP

Description: Le systeme convertit les capacites en propositions de tools MCP: nom, description courte, parametres, schema, exemple d'appel et exemple de retour. Les descriptions doivent etre claires, compactes et utiles pour un agent IA.

Valeur utilisateur: Obtenir des tools utilisables par Claude, ChatGPT, Cursor ou un autre client MCP.

Criteres d'acceptation MVP:

- Schema d'entree valide
- Description non vague
- Exemple d'utilisation genere

F5 - Security scoring et permissions

Description: Chaque tool recoit un niveau de risque. Le systeme signale les actions sensibles: ecriture en base, suppression, execution de commandes, lecture de secrets, operations reseau, modification de fichiers ou actions irreversibles.

Valeur utilisateur: Eviter de generer un serveur MCP dangereux par default.

Criteres d'acceptation MVP:

- Score visible dans l'interface
- Actions dangereuses desactivees par default
- Justification de risque lisible

F6 - Validation humaine avant generation

Description: L'utilisateur voit les tools proposes, peut en activer/desactiver, renommer un tool, modifier sa description et retirer les operations sensibles.

Valeur utilisateur: Garder le controle produit et securite.

Criteres d'acceptation MVP:

- Aucun tool write/destructive n'est genere sans validation explicite
- Preview claire avant generation
- Configuration sauvegardee dans le projet

F7 - Generation du serveur MCP TypeScript

Description: Le generateur produit un serveur MCP TypeScript avec structure propre: src/server.ts, src/tools, src/lib, schemas, gestion d'erreurs, .env.example et scripts npm.

Valeur utilisateur: Recevoir un projet qui peut etre lance localement sans repartir de zero.

Criteres d'acceptation MVP:

- npm install fonctionne
- npm run build fonctionne
- Serveur MCP demarre en mode stdio
- Tools declares proprement

F8 - Generation de documentation et config client

Description: Le systeme genere un README clair avec installation, variables d'environnement, liste des tools, exemples d'appels et exemples de configuration pour des clients MCP courants.

Valeur utilisateur: Rendre le serveur partageable et installable rapidement.

Criteres d'acceptation MVP:

- README complet
- Config client presente
- Chaque variable d'environnement expliquee

F9 - Validation technique automatique

Description: Apres generation, le systeme lance des checks: format, typecheck, tests de schemas, verification que les tools sont bien declares et que les imports existent.

Valeur utilisateur: Eviter de livrer un ZIP casse.

Criteres d'acceptation MVP:

- Rapport de validation visible
- Erreurs classees
- Patch automatique tente une seule fois dans le MVP

F10 - Export ZIP

Description: Une fois valide, le projet genere est package dans un ZIP telechargeable contenant le serveur MCP, la documentation et les fichiers de configuration.

Valeur utilisateur: Permettre au developpeur de tester localement immediatement.

Criteres d'acceptation MVP:

- ZIP genere
- Nom du projet propre
- Aucun fichier temporaire ni secret inclus

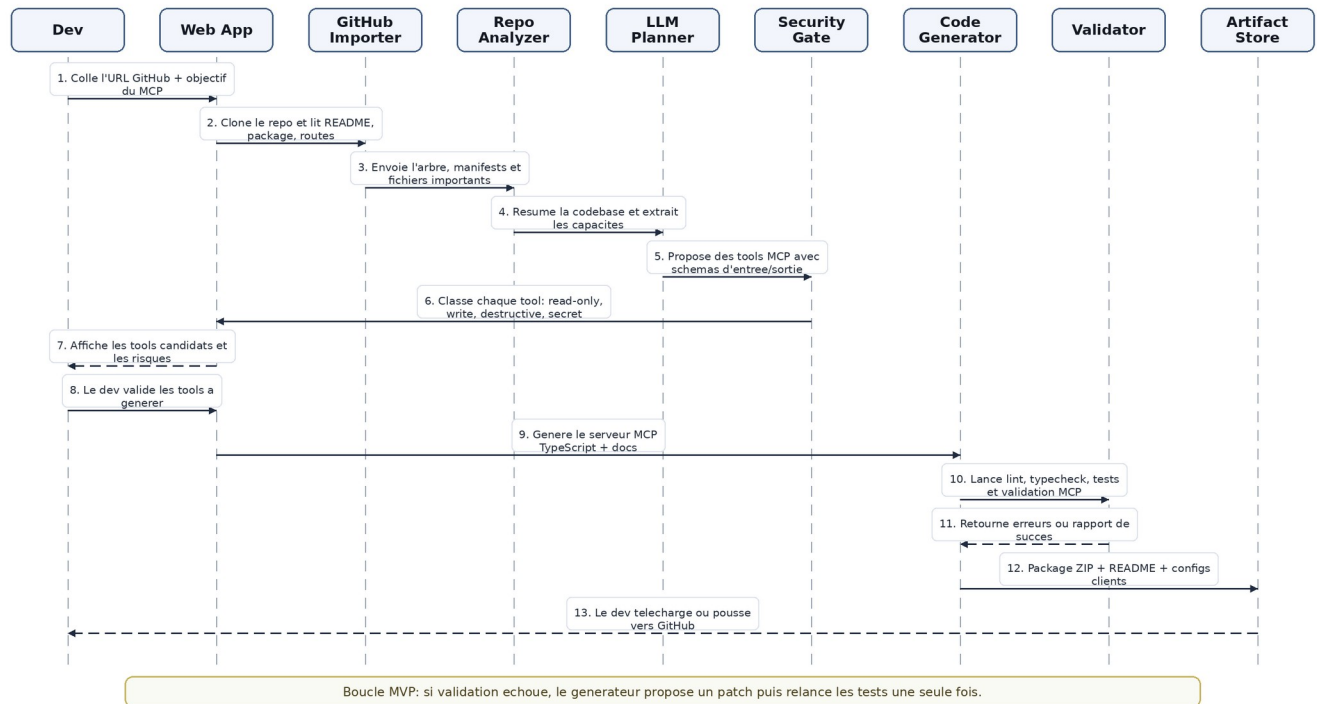
4. Parcours utilisateur MVP

Etape	Nom	Description
1	Input	Le developpeur colle l'URL du repo GitHub et choisit le type de serveur MCP souhaite: read-only, mixed ou experimental.
2	Analyse	MCP Maker lit le repo, detecte le stack, extrait les routes/services/scripts et produit un resume technique.
3	Proposition	L'app montre les tools MCP candidats avec schema, description, exemple et niveau de risque.
4	Validation	Le developpeur active uniquement les tools utiles et corrige les noms/descriptions si necessaire.
5	Generation	Le systeme genere le serveur MCP, la documentation, les tests et les configs client.
6	Verification	Le systeme lance build/typecheck/tests et produit un rapport. En cas d'erreur simple, il tente une correction.
7	Export	Le developpeur telecharge le ZIP et lance le serveur localement dans son client MCP.

5. Diagramme de sequence du MVP

Ce diagramme montre le flux principal, depuis l'import GitHub jusqu'a l'export du serveur MCP genere.

Diagramme de sequence - MVP MCP Maker



6. Architecture logique proposee

Bloc	Technologie possible	Responsabilite
Frontend	Next.js ou React	Dashboard: URL repo, preview tools, scoring, generation et export.
API Backend	Node.js/TypeScript	Orchestration: jobs, analyse, generation, validation, packaging.
Repo Worker	Sandbox temporaire	Clone, lecture fichiers, extraction tree, nettoyage final.
Analyzer	AST + heuristiques	Detection stack, routes, services, schemas, scripts, env vars.
LLM Planner	Agent de planification	Resume la codebase et propose des tools MCP lisibles.
Security Engine	Rules engine	Score les tools, bloque les operations dangereuses par default.
Generator	Templates + codegen	Produit serveur MCP TypeScript, schemas, docs et tests.
Validator	CI locale	npm install, build, typecheck, tests, rapport.
Storage	S3/local	Stocke temporairement ZIP, rapports et configuration de generation.

7. Objets metier essentiels

Modele	Champs principaux
ProjectAnalysis	repoUrl, provider, detectedStack, fileTree, importantFiles, summary, confidenceScore
CapabilityCandidate	name, sourceFile, type, description, inputHints, outputHints, sideEffects
McpToolDraft	toolName, description, inputSchema, outputExample, riskLevel, enabled
SecurityFinding	toolName, severity, reason, recommendation, defaultAction
GenerationJob	projectId, selectedTools, status, logs, artifactUrl, validationReport

8. Regles de securite MVP

MCP Maker doit etre security-first. Un serveur MCP peut exposer des actions puissantes a un agent IA. Le MVP doit donc assumer que toute generation automatique est potentiellement risquee.

- Mode read-only par default pour la premiere generation.
- Blocage par default des tools qui suppriment, executent des commandes shell ou modifient des fichiers systeme.
- Detection basique des secrets: .env, tokens, keys, credentials, connection strings.
- Aucun secret du repo source n'est copie dans le serveur genere.

- Chaque tool sensible doit avoir une justification et une validation manuelle.
- Logs d'analyse sans dump complet de fichiers sensibles.
- Rapport clair: pourquoi un tool est considere dangereux ou acceptable.

9. Roadmap conseillée

Phase	Objectif
Phase 1 - Prototype local	Importer un repo JS/TS, produire un resume, proposer 5 a 10 tools candidats.
Phase 2 - Generation TypeScript	Generer un serveur MCP stdio avec schemas, README et tests basiques.
Phase 3 - Interface review	Dashboard pour selectionner, renommer, activer et desactiver les tools.
Phase 4 - Security scoring	Regles de risque, mode read-only par default, blocage des operations dangereuses.
Phase 5 - Validation runner	Build, typecheck, tests, rapport d'erreurs et tentative de patch simple.
Phase 6 - Export	ZIP telechargeable, templates client, historique de generation.

10. Limites assumées du MVP

- Le MVP ne doit pas promettre une generation parfaite pour tous les repos.
- Les repos mal structures ou sans types clairs produiront des propositions moins fiables.
- La premiere version doit privilegier TypeScript/Node pour limiter la complexite.
- Les operations write/destructive doivent rester experimentales et explicites.
- Le systeme peut proposer des tools, mais le developpeur reste responsable de ce qui est expose.

11. Indicateurs de succes

Metric	Definition
Time-to-first-server	Temps entre l'URL GitHub et un ZIP MCP lançable localement.
Build success rate	Pourcentage de projets generes qui passent npm run build.
Tool relevance	Pourcentage de tools proposes que l'utilisateur garde.
Security usefulness	Nombre de tools dangereux correctement detectes avant generation.
Manual edits needed	Nombre de corrections faites par l'utilisateur apres export.

12. References utiles

- Model Context Protocol - Introduction: <https://modelcontextprotocol.io/docs/getting-started/intro>
- MCP Specification - Tools: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- GitHub official MCP Server: <https://github.com/github/github-mcp-server>